



Statistical Methods & Machine Learning 1

Class #14

John Kubale

December 3, 2024

Outline

- Resampling
 - Bootstrap
 - Cross-validation
- Review
- Lab



Resampling

Resampling

- What is resampling?
 - Repeatedly sampling from a (training) dataset and refitting a model on each sample
- Why resampling?
 - Assessing parameter estimate precision: e.g., standard error, confidence interval
 - Cross-validation
 - Model assessment: model fit and performance (e.g., MSE)
 - Model selection: e.g., model flexibility
- Resampling methods
 - Bootstrap
 - Cross-validation: Validation set approach, leave-one-out (LOO), k -fold cross-validation



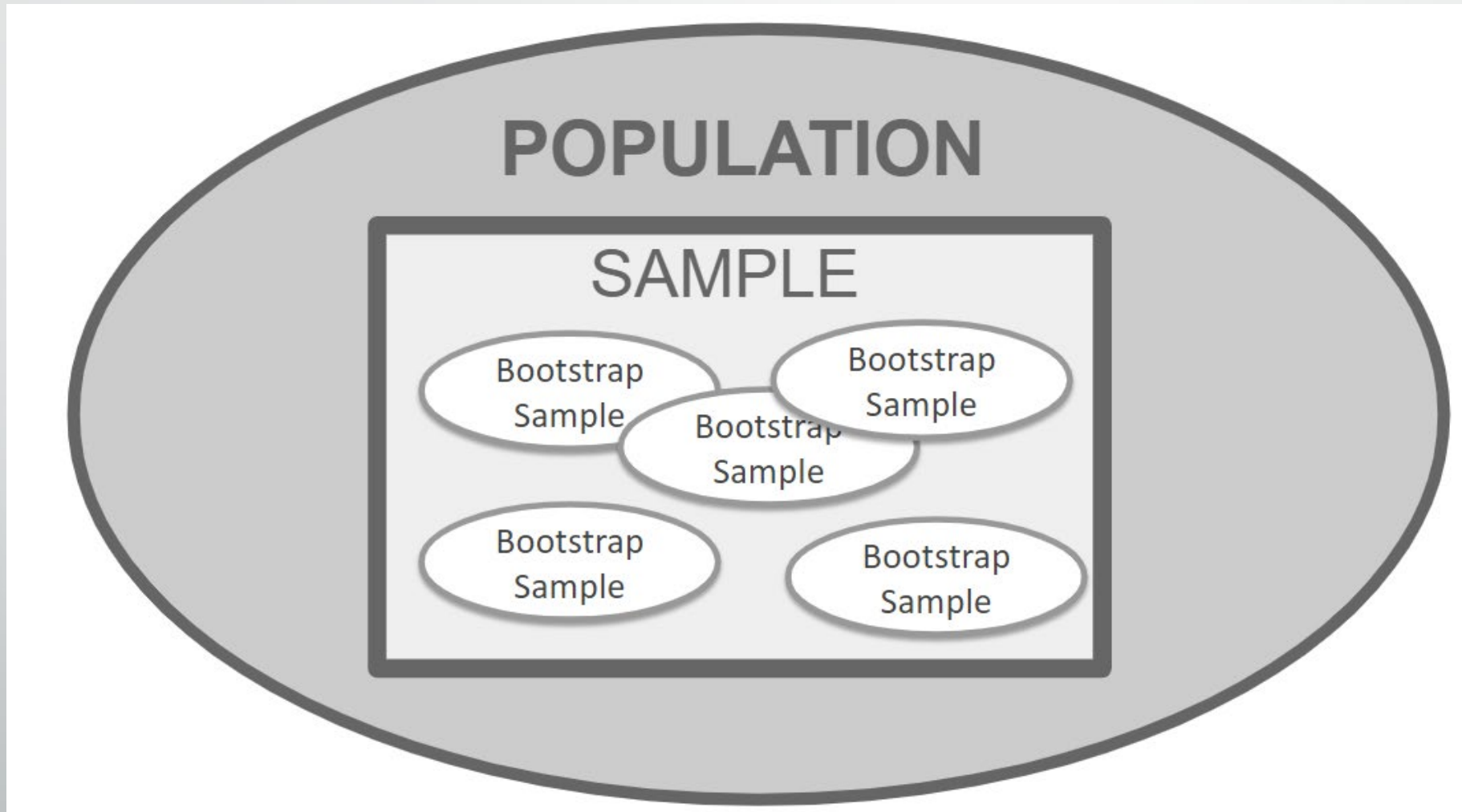
Bootstrap

Bootstrap - 1

- Why?
 - What if the OLS assumptions in linear regression are violated and the remedies we have learned so far do not resolve the problem(s)?
- How?
 - Say that we are interested in a parameter θ
 - Draw a sample of n with replacement from the original sample of n repeatedly, say B times
 - Observe the estimate in each sample, e.g., $\hat{\theta}_r$, where $r = 1, \dots, B$
 - Combine the results $\hat{\theta}_1, \dots, \hat{\theta}_B$ as follows:

$$SE_B \hat{\theta} = \sqrt{\frac{1}{B-1} \sum_{r=1}^B (\hat{\theta}_r - \frac{1}{B} \sum_{r=1}^B \hat{\theta}_r)^2}$$

Bootstrap - 2

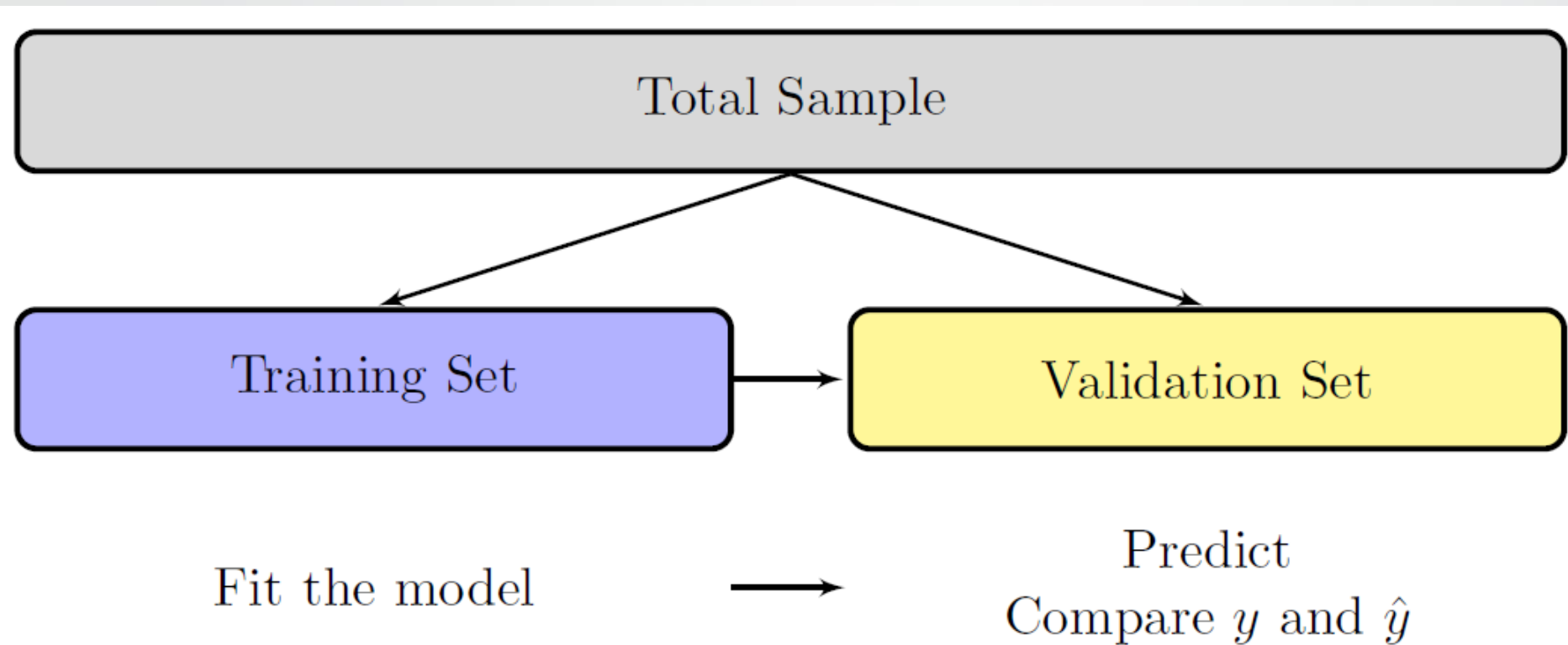


Bootstrap - 3

- Alternatively for linear regression (in Faraway)
 1. Generate ϵ^* from $\hat{\epsilon}_1, \dots, \hat{\epsilon}_n$
 2. Generate $y^* = X\hat{\beta} + \epsilon^*$
 3. Estimate $\hat{\beta}^*$ by modeling y^* on X
 4. Repeat and observe $\hat{\beta}^*$
 5. Compute $SE(\beta)$ by summarizing $\hat{\beta}^*$



Cross-validation



How to split the sample?

1. Validation set approach
2. Leave-one-out
3. k -fold

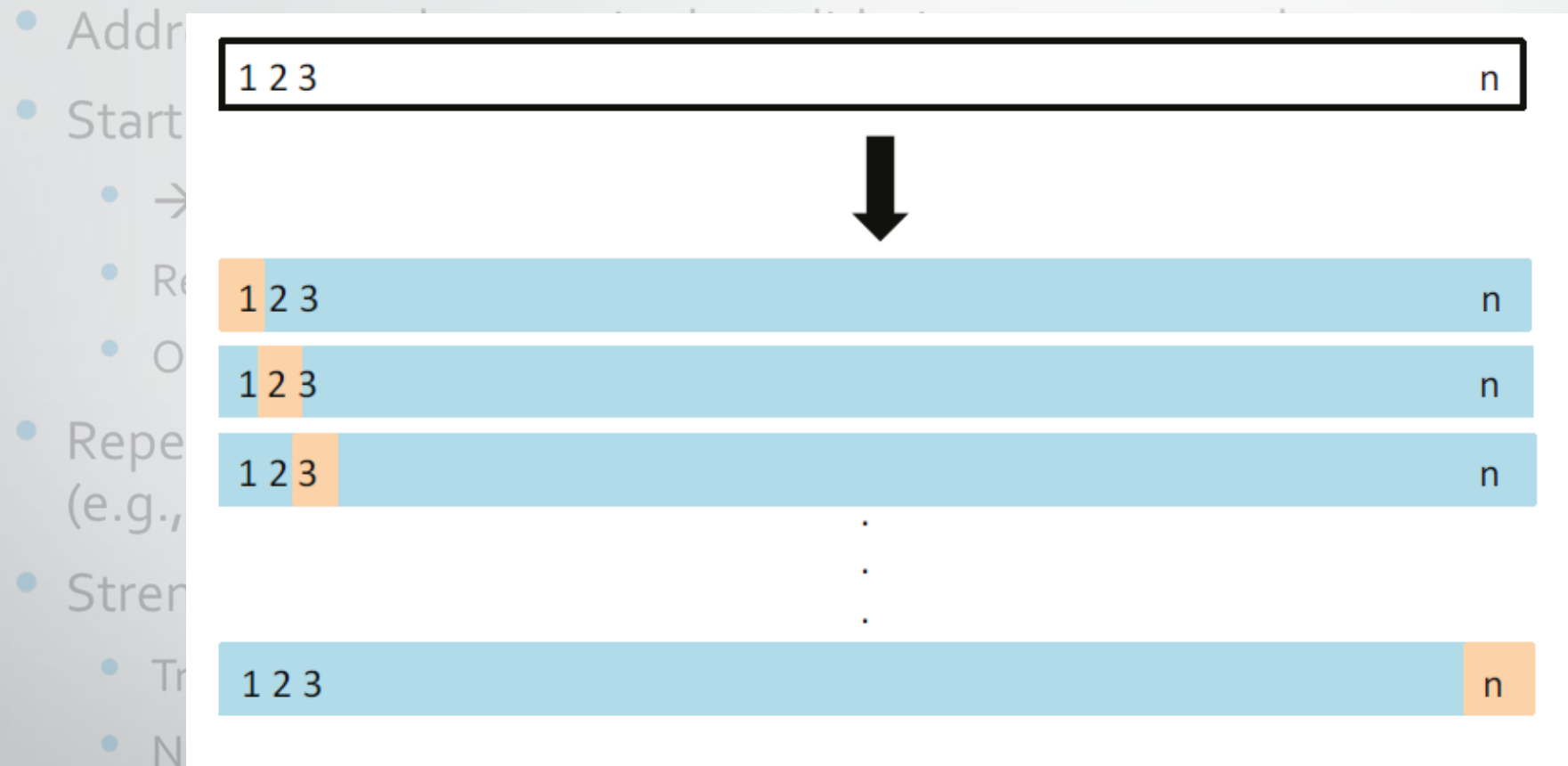
Cross-validation: Validation Set Approach

- Conceptually and operationally simple
- Split the sample randomly in half (or into 2 groups with different proportions)
 - Use one as a training set
 - The other as a validation set and observe the test error rate
(recall, $\epsilon = y - \hat{y}$)
- Weaknesses
 - High variability in validation estimates (depending on the single random split)
 - Overestimation of error rates (because statistical methods perform worse when trained on fewer observations)

Cross-validation: Leave-One-Out (LOO)

- Addresses weaknesses in the validation set approach
- Start from the 1st observation in the sample
 - → Validation set
 - Remaining observations in the original sample → Training set
 - Observe the test error in the validation set
- Repeat steps above for all n observations and summarize the test error rates (e.g., average)
- Strengths over validation set approach
 - Trained on more observations → less bias
 - No randomness in sample split
- Weaknesses
 - Depending on the model and sample size can be very computationally intensive

Cross-validation: Leave-One-Out (LOO)



ISLR Figure 5.3

- Addressing the bias of training error rates
- Start with a small sample size
 - $\rightarrow$ Compute the training error rate
 - Repeat for different sample sizes
 - Observe how the training error rate changes
- Repeat the process for different sample sizes (e.g., $n=10, 20, 30, \dots$)
- Strengthening the model
 - Training error rate decreases as sample size increases
 - Test error rate initially decreases and then increases
- Weaknesses
 - Depending on the model and sample size can be very computationally intensive

Cross-validation: k -Fold

- Alternative to Leave-One-Out
- Divide the sample into k folds (i.e., subsets)
 - $\rightarrow$ 1st fold is a validation set
 - Remaining folds in the original sample $\rightarrow$ training set
- Repeat steps above over k folds
- k is usually 5-10
- Strengths
 - Less computationally intensive
 - Advantageous in bias-variance trade off vs. LOO
 - Slightly more biased than LOO
 - Smaller variance than LOO where training sets are almost identical ($\rightarrow$ Highly correlated fitted models $\rightarrow$ Adds to variability)

Cross-validation: k -Fold

- Alternative to Leave One Out

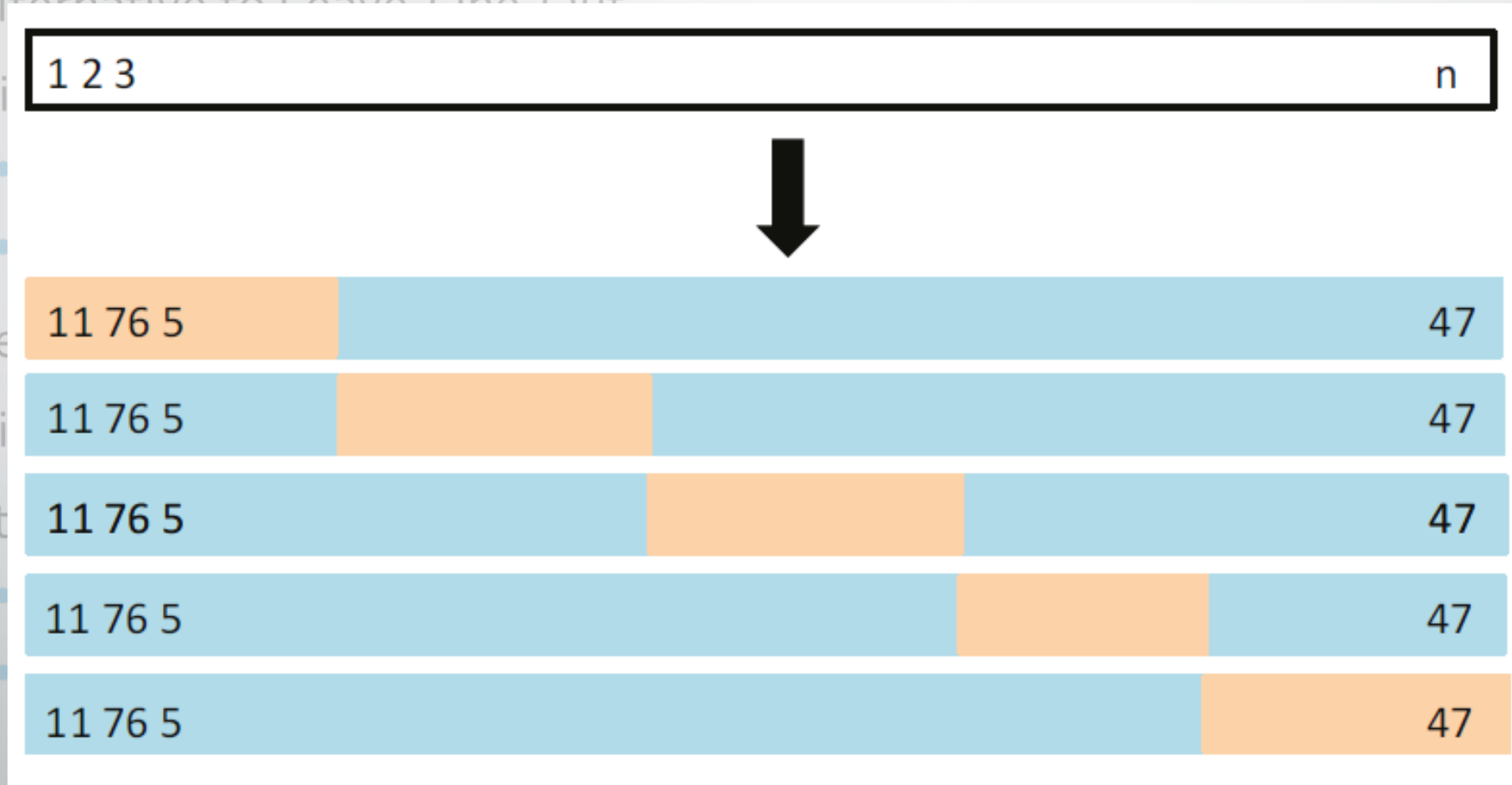
- Di

-

- Re

- k i

- St



- Smaller variance than LOO where training sets are almost identical (only 1 correlated fitted models → Adds to variability) ISLR Figure 5.5

Review – Regression terminology

- The model $\text{lm}(y \sim x + z, \text{data})$ can be described as:
 - Regress y on x and z
 - Fit a linear model of y as a function of x and z

Review - Assessing collinearity

- Eigenvalues
- Condition number
- Kappa
- Variance Inflation Factor

Review - Assessing collinearity

- Eigenvalues
- Condition number
- Kappa
- Variance Inflation Factor

Review - Assessing collinearity

- Eigenvalues → eigenvalue of 0 means perfect collinearity
- Condition number → $\sqrt{\max(\text{eigenvalue}) / \text{eigenvalue}_i}$
- Kappa → $\sqrt{\max(\text{eigenvalue}) / \min(\text{eigenvalue})}$
- Variance Inflation Factor → want to avoid $VIF > 5$ or 10 in social sciences

Review - Assessing collinearity

High condition numbers or kappa do not mean there is definitely collinearity

- Eigenvalues → eigenvalue of 0 means perfect collinearity
- Condition number → $\sqrt{\max(\text{eigenvalue}) / \text{eigenvalue}_i}$
- Kappa → $\sqrt{\max(\text{eigenvalue}) / \min(\text{eigenvalue})}$
- Variance Inflation Factor → want to avoid $VIF > 5$ or 10 in social sciences

Review

- What are our linear regression assumptions?
- How do we assess whether these assumptions are met?
- What can we do if the assumptions are not met?

Zero Mean Error

$$E(\epsilon_i) \neq 0$$

What is the problem?

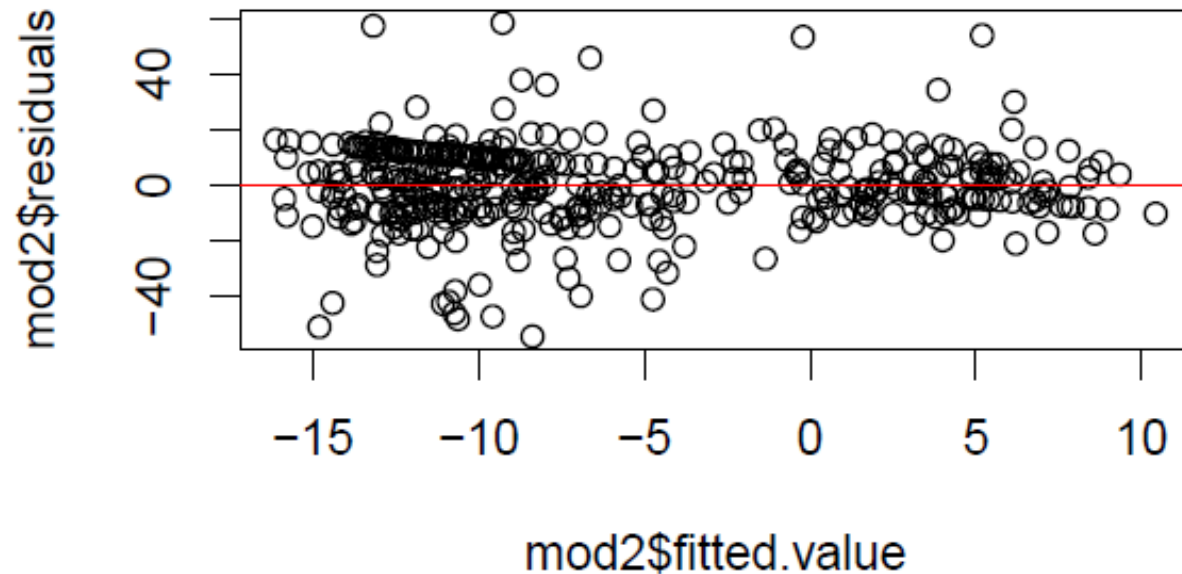
- OLS estimators are not unbiased
(e.g., Cannot guarantee $E(\hat{\beta}_{OLS}) = \beta$)

What to do?

- Include other predictors: e.g., splines, polynomials, interactions
- Variable transformation

Assessing Zero Mean Error

```
plot(mod2$fitted.value, mod2$residuals); abline(h=0, col="red")
```



What do you see?

Assessing Zero Mean Error

```
mean(resid(mod2));mean(resid(mod2)[fitted(mod2)<=-5]);mean(resid(mod2)[fitted(mod2)>-5])
```

```
## [1] 2.678447e-16
```

```
## [1] 0.009675334
```

```
## [1] -0.01612556
```


Assessing Zero Mean Error

```
mean(resid(mod2));mean(resid(mod2)[fitted(mod2)<=-5]);mean(resid(mod2)[fitted(mod2)>-5])
```

```
## [1] 2.678447e-16
```

```
## [1] 0.009675334
```

```
## [1] -0.01612556
```

What assumption(s)
should we check prior to
conducting a t test?

```
t.test(resid(mod2)[fitted(mod2)<=-5],resid(mod2)[fitted(mod2)>-5])
```

```
##
```

```
## Welch Two Sample t-test
```

```
##
```

```
## data: resid(mod2)[fitted(mod2) <= -5] and resid(mod2)[fitted(mod2) > -5]
```

```
## t = 0.018077, df = 373.92, p-value = 0.9856
```

```
## alternative hypothesis: true difference in means is not equal to 0
```

```
## 95 percent confidence interval:
```

```
## -2.780756 2.832358
```

```
## sample estimates:
```

```
## mean of x mean of y
```

```
## 0.009675334 -0.01612557
```

Homoscedasticity (i.e., Constant Variance)

$$V(\epsilon_i) = \sigma^2 \text{ vs. } V(\epsilon_i) = \sigma_i^2$$

What is the problem?

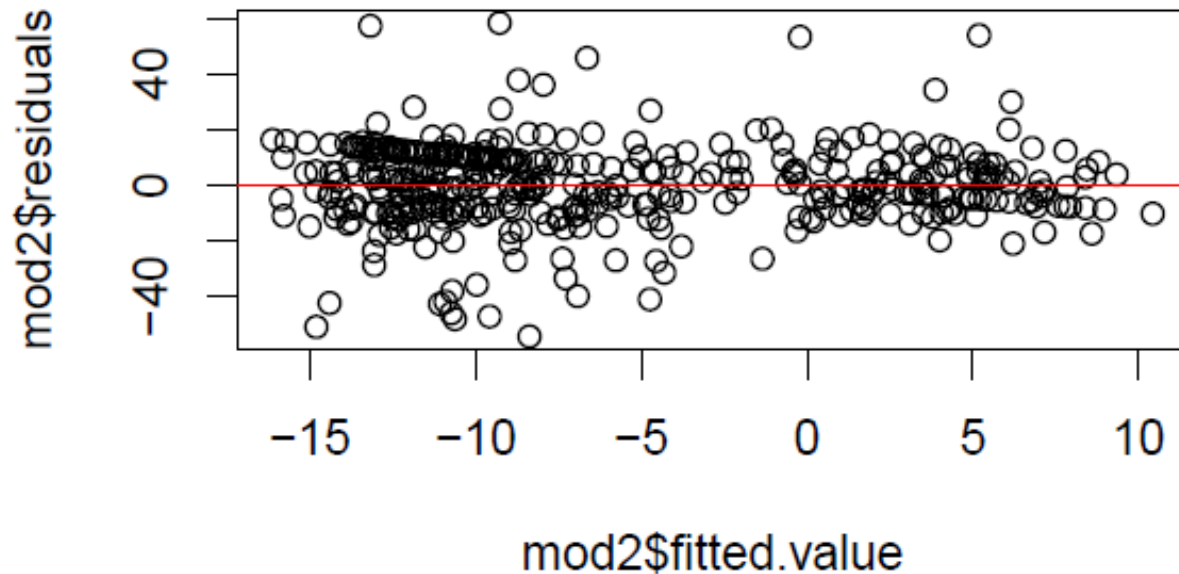
- Cannot guarantee that $\hat{V}(\hat{\beta})$ is unbiased of $V(\hat{\beta})$

What to do?

- Heteroscedastic consistent variance estimator
- Weighted least squares estimation
- Variable transformation

Assessing Homoscedasticity

```
plot(mod2$fitted.value, mod2$residuals); abline(h=0, col="red")
```



What do you see?

Assessing Homoscedasticity

```
var(resid(mod2));var(resid(mod2)[fitted(mod2)<=-5]);var(resid(mod2)[fitted(mod2)>-5])
```

```
## [1] 216.9507
```

```
## [1] 251.3609
```

```
## [1] 160.8764
```

```
var.test(resid(mod2)[fitted(mod2)<=-5],resid(mod2)[fitted(mod2)>-5])
```

```
##
```

```
## F test to compare two variances
```

```
##
```

```
## data: resid(mod2)[fitted(mod2) <= -5] and resid(mod2)[fitted(mod2) > -5]
```

```
## F = 1.5624, num df = 254, denom df = 152, p-value = 0.002756
```

```
## alternative hypothesis: true ratio of variances is not equal to 1
```

```
## 95 percent confidence interval:
```

```
## 1.168766 2.067245
```

```
## sample estimates:
```

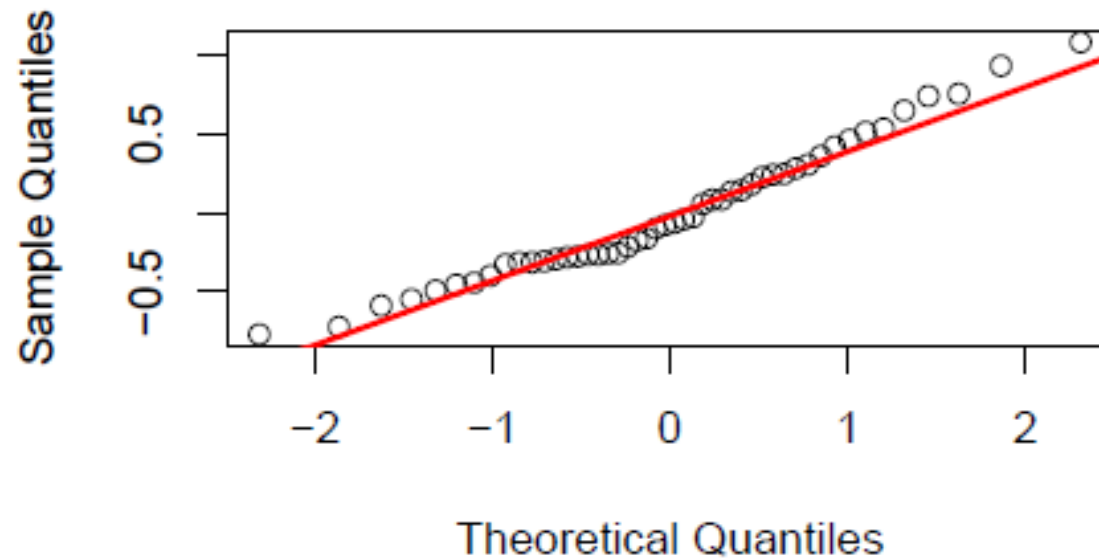
```
## ratio of variances
```

```
## 1.562447
```

Normality of Errors

```
qqnorm(tmod$residual)  
qqline(tmod$residual, col = "red", lwd = 2)
```

Normal Q-Q Plot





Normality of Errors

How might we address violations of this assumption?

Normality of Errors

How might we address violations of this assumption?

- Transformation
- GAMs or polynomial regression models
- Bootstrapping our standard errors

Random effects models

```
library(lme4)
rmod<-lmer(time~(1|treat), rats)
summary(rmod)

## Linear mixed model fit by REML ['lmerMod']
## Formula: time ~ (1 | treat)
## Data: rats
##
## REML criterion at convergence: -0.5
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -1.6369 -0.7218 -0.1514  0.5186  2.7284
##
## Random effects:
## Groups   Name      Variance Std.Dev.
## treat    (Intercept) 0.02164  0.1471
## Residual                    0.04736  0.2176
## Number of obs: 48, groups:  treat, 4
##
## Fixed effects:
##              Estimate Std. Error t value
## (Intercept)  0.47937    0.07998   5.993
```

Can we make any conclusions about the hypothesis being tested from the summary() output?

Random effects models

```
confint(rmod, oldNames = FALSE)
```

```
##                2.5 %    97.5 %  
## sd_(Intercept)|treat 0.04474072 0.3338817  
## sigma                0.17900826 0.2724161  
## (Intercept)         0.30345222 0.6552976
```

No, we need to look
at the confidence
interval.

Random effects models

```
confint(rmod, oldNames = FALSE)
```

```
##                2.5 %    97.5 %  
## sd_(Intercept)|treat 0.04474072 0.3338817  
## sigma                0.17900826 0.2724161  
## (Intercept)         0.30345222 0.6552976
```

No, we need to look
at the confidence
interval.

- `sd_(Intercept)|treat` we can see that the 95% CI does not include 0. This suggests that we can reject our null hypothesis that $\sigma_a = 0$.
(Lecture 13)



Lab Session

For loops

- **For loops** are commonly used to help streamline repetitive tasks when coding
- Structure of for loop:

```
for (item in list of items){  
    do_something(item)  
}
```

For loops

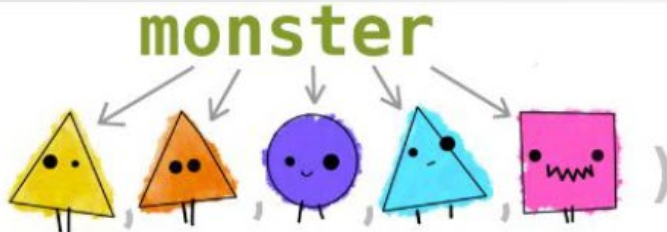
- **For loops** are commonly used to help streamline repetitive tasks when coding

- Structure of for loop:

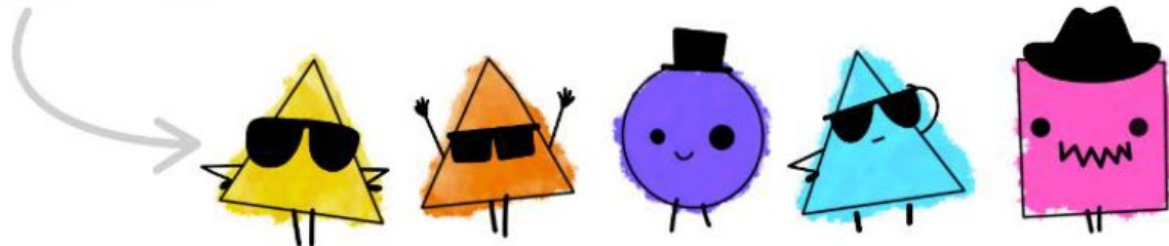
```
for (item in list of items) {  
    do_something(item)  
}
```

- **In words:** For each item in a sequence or list of items (vector, dataframe, etc.), perform some operation (apply a function)

For loops

`parade = c(` `)`

```
for (monster in parade) {  
    if (shape(monster) == triangle) {  
        monster_style = monster + sunglasses  
    }  
    else {  
        monster_style = monster + hat  
    }  
    print(monster_style)  
}
```



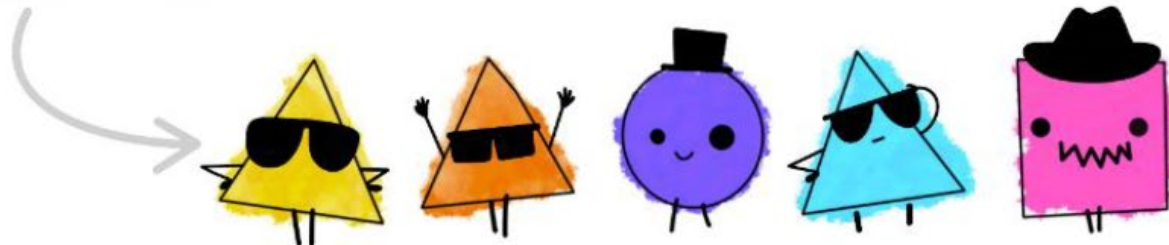
For loops

`parade = c(`  `,`  `,`  `,`  `,`  `)`

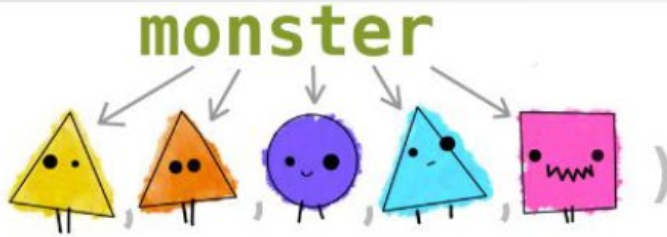
The word `monster` is written above the first four monsters with arrows pointing to each of them.

There are 5 elements in parade, so R will iterate the commands in the for loop 5 times, once for each element in parade.

```
for (monster in parade) {  
  if (shape(monster) == triangle) {  
    monster_style = monster + sunglasses  
  }  
  else {  
    monster_style = monster + hat  
  }  
  print(monster_style)  
}
```



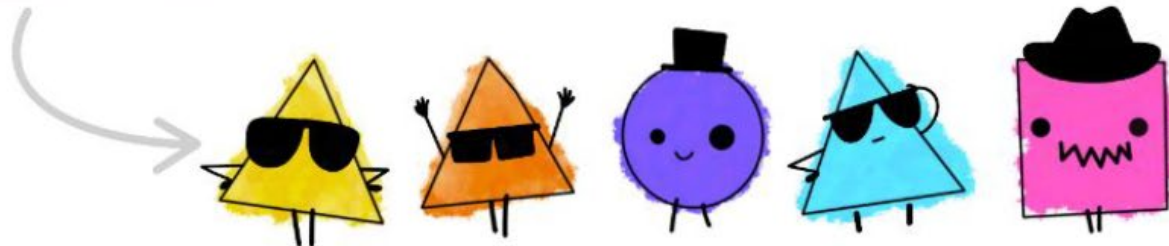
For loops

`parade = c(`  `)`

There are 5 elements in parade, so R will iterate the commands in the for loop 5 times, once for each element in parade.

```
for (monster in parade) {  
  if (shape(monster) == triangle) {  
    monster_style = monster + sunglasses  
  }  
  else {  
    monster_style = monster + hat  
  }  
  print(monster_style)  
}
```

Uses a conditional statement (IF/ELSE) to determine the values of monster_style depending on the values in monster



For loops

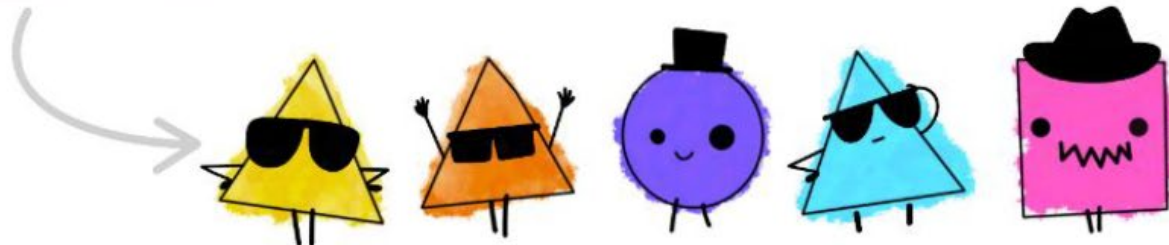
`parade = c(`  `,`  `,`  `,`  `,`  `)`

There are 5 elements in parade, so R will iterate the commands in the for loop 5 times, once for each element in parade.

```
for (monster in parade) {  
  if (shape(monster) == triangle) {  
    monster_style = monster + sunglasses  
  }  
  else {  
    monster_style = monster + hat  
  }  
  print(monster_style)  
}
```

Uses a conditional statement (IF/ELSE) to determine the values of monster_style depending on the values in monster

Tells R to print monster_style.



For loops

What does this code do?

```
48 ▾ ```{r}
49   mean_gamble<-rep(0,5)
50
51 ▾   for(i in 1:5){
52     samp_gamble<-sample(gamble,5,replace=T)
53     mean_gamble[i]<-mean(samp_gamble)
54 ▴   }
55 ▴ ```
```

For loops

What does this code do?

Creates vector called "mean_gamble" consisting of 0 repeated five times.

```
48 ▾ ```{r}
49 mean_gamble<-rep(0,5)
50
51 ▾ for(i in 1:5){
52     samp_gamble<-sample(gamble,5,replace=T)
53     mean_gamble[i]<-mean(samp_gamble)
54 ▸ }
55 ▸ ```
```

For loops

What does this code do?

Creates vector called "mean_gamble" consisting of 0 repeated five times.

```
48. {r}
49. mean_gamble<-rep(0,5)
50.
51. for(i in 1:5){
52.   samp_gamble<-sample(gamble,5,replace=T)
53.   mean_gamble[i]<-mean(samp_gamble)
54. }
55.
```

For each item i in a list from 1:5 do the operations inside the curly braces.

For loops

What does this code do?

Creates vector called "mean_gamble" consisting of 0 repeated five times.

```
48.  ```{r}
49.  mean_gamble<-rep(0,5)
50.
51.  for(i in 1:5){
52.    samp_gamble<-sample(gamble,5,replace=T)
53.    mean_gamble[i]<-mean(samp_gamble)
54.  }
55.  ```
```

Take 5 samples from "gamble" (with replacement) and store in vector called "samp_gamble".

For each item i in a list from 1:5 do the operations inside the curly braces.

For loops

What does this code do?

```
48 • ```{r}
49 mean_gamble<-rep(0,5)
50
51 • for(i in 1:5){
52   samp_gamble<-sample(gamble,5,replace=T)
53   mean_gamble[i]<-mean(samp_gamble)
54 • }
55 • ```
```

Creates vector called "mean_gamble" consisting of 0 repeated five times.

Take 5 samples from "gamble" (with replacement) and store in vector called "samp_gamble".

For each item i in a list from 1:5 do the operations inside the curly braces.

Take mean of "samp_gamble" and store in position corresponding to iteration number (1-5)

For loops

What does this do?

Creates vector called
"mean_gamble"
consisting of 0 repeated

You end up with the 0's in
mean_gamble being replaced by the
mean of each of the 5 samples drawn.

```
48 ~~~{r}  
49 mean_gamble  
50  
51 for(i in 1:  
52   samp_gamb  
53   mean_gamb  
54 }  
55 ~~~
```

```
> mean_gamble  
[1] 30.98  9.56  2.82 42.30  4.94  
>
```

Take 5 samples from
"samp_gamble" and store in
"samp_gamble".

For each item i in a list
from 1:5 do the
operations inside the
curly braces.

Take mean of
"samp_gamble" and store
in position corresponding
to iteration number (1-5)

Functions

- Have used existing functions all semester long (e.g., `mean()`, `lm()`, `confint()`)
- Can also define our own functions to improve efficiency of our code using `function() { }`

Functions

How to create
a custom function in 

```
Function  
name    "function" & curly  
         ↓          brackets  
add_three <- function(arg){  
    output <- arg + 3  
    return(output)  
}
```

arguments

Function
body

Functions

How to create
a custom function in 

```
> add_three <- function(arg){  
+   output = arg + 3  
+   return(output)  
+ }  
> add_three(3)  
[1] 6  
> |
```

↑
Function
body

arguments

Functions

Where we
define/create the
function "add_three".

How to create
a custom function in 

```
> add_three <- function(arg){  
+   output = arg + 3  
+   return(output)  
+ }  
  
> add_three(3)  
[1] 6  
> |
```

↑
Function
body

arguments

Functions

How to create
a custom function in 

```
> add_three <- function(arg){  
+   output = arg + 3  
+   return(output)  
+ }
```

```
> add_three(3)  
[1] 6  
> |
```

Where we call/use the
function "add_three".

↑
Function
body

arguments

Lab Session

- Use diabetes from faraway
- Focus on glyhb as a random variable and a model $\text{glyhb} \sim \text{chol}$
- Bootstrap
 - Mean of a random variable and its standard error and confidence interval
 - Simple linear regression: Standard error and confidence interval of regression coefficient
- Cross-validation in linear regression
 - Validation set approach
 - LOO
 - k -fold